

BNS Warehouse System

Setup Guide

Everything needed to take a fresh copy of this system from zero to a fully configured, day-one-ready install: what to install, first run, every setting that needs populating (DPD, email, Shopify, printing), and the two bulk import files - what they contain, where they go, and what they add.

Document version: 1.0 | Covers: v0.095 | Last updated: 2026-09-24

This is an operational runbook, not a feature tour - see the separate User Guide for how to use the screens day to day, and the two API PDFs for the public Stock and Support Tickets APIs.

Contents

Contents	2
1. Prerequisites - What Needs Installing	4
1.1 On the server/PC that will run the system	4
1.2 Optional - only if these are needed on day one	4
2. First-Time Setup	4
2.1 Unpack and configure	4
2.2 Build and start	4
2.3 First login	5
3. Settings Walkthrough	5
3.1 Printing	5
3.2 DPD	5
3.3 Email	5
3.4 Despatch & Packing	5
3.5 Returns (RMA)	6
3.6 Users	6
3.7 My Email	6
3.8 Customisation	6
4. Shopify Setup	6
4.1 Create a Shopify app and get credentials	6
4.2 Configure and connect	6
4.3 What each sync does	7
5. DPD Setup	7

5.1 API credentials	7
5.2 Sender / collection address	8
5.3 Customs declaration	8
5.4 Extended liability and fallback labels	8
6. Email Setup	8
6.1 Settings → Email (shared/fallback)	9
6.2 Settings → My Email (per-person, optional)	9
7. Printing (Print Agent) Setup	9
7.1 Which way to run it	9
7.2 Setup steps	9
7.3 Print Agent Troubleshooting	10
8. Handheld / Android Kiosk (Optional)	11
9. Bulk Import Files	11
9.1 Bulk Stock Import	11
9.2 Company & Contact Import	12
9.3 API keys (not a bulk import, but a related one-off setup step)	13
10. Backup & Restore	13
10.1 Taking a backup	13
10.2 Standing up a brand new machine from a backup	14
10.3 Restoring into an already-running system	14
11. Before Going Fully Live - Known Gaps	14

1. Prerequisites - What Needs Installing

This system runs entirely inside Docker containers, so almost nothing needs installing by hand on the server itself - no Java, no Node, no local PostgreSQL install.

1.1 On the server/PC that will run the system

- **Docker Desktop** (Windows/Mac) or **Docker Engine + Docker Compose** (Linux) - this is the only real prerequisite. Everything else (Java 21, Node, PostgreSQL 17) is bundled inside the containers Docker builds.
- A machine that can stay on and reachable on the local network during working hours - this is built and tested as a LAN-only tool (see Section 10).
- Ports free on that machine: **8081** (the web app), **8080** (the API - also used directly for the Shopify OAuth handshake, see Section 4), and **5432** (PostgreSQL).

1.2 Optional - only if these are needed on day one

- **The Print Agent** (Section 7) - a small Python program that runs on the PC actually doing the printing (the despatch bench), so picking notes and DPD labels print silently with no dialog box. Needs Python, SumatraPDF, and (for DPD labels) the pywin32 package - all on that PC specifically, not the server.
- **Android Studio** - only needed if building the handheld kiosk Android app (android-app/). The handheld scanner screens work fine in any regular browser without this; it's only for a dedicated, locked-down Android scanner device.

Note: Nothing above needs an internet connection once the Docker images are built, except: Shopify sync (talks to Shopify's API), sending real email (talks to your SMTP provider), and booking real DPD shipments (talks to DPD's API). The system runs fully offline otherwise.

2. First-Time Setup

2.1 Unpack and configure

- 1 Unzip the release into a folder on the server (e.g. bns-warehouse-system/).
- 2 Copy .env.example to .env in the project root.
- 3 Open .env and set real values for POSTGRES_DB, POSTGRES_USER and POSTGRES_PASSWORD (or leave the example defaults for a first test run - just don't leave them as-is anywhere internet-facing).
- 4 Leave SMTP_* and SHOPIFY_* blank for now if those aren't ready yet - the system runs fine without them and both can be filled in later with no data loss (see Sections 4 and 6).

2.2 Build and start

From the project root:

```
docker compose up --build
```

First run downloads and builds everything (Maven dependencies, npm packages) so it takes a few minutes. Every run after that is much faster, since Docker caches the unchanged layers. Once it's up, Flyway creates the full database schema automatically - nothing to run by hand.

2.3 First login

- **Frontend (the main app):** `http://<server>:8081` (use `localhost:8081` if browsing from the server itself)
- **Handheld scanner app:** `http://<server>:8081/handheld`
- **Public RMA return form** (no login needed): `http://<server>:8081/rma`
- **API** (for integrations, not for browsing): `http://<server>:8080/api`

The very first startup seeds one login: **admin**, password **ChangeMe123!**. Log in with that, then go straight to **Settings** → **Users** and change the password (or add real named logins for everyone and stop using the seeded one) - see Section 3.5. Every screen except the public RMA form requires a login.

Tip: Replace `<server>` above with the server's actual LAN IP address (e.g. `192.168.1.245`) for anyone accessing it from another PC - `localhost` only works from the server itself. If that IP can change (DHCP), either set a static/reserved IP for the server in the router, or everyone's bookmarks (and the Android kiosk app, Section 8) will need updating whenever it moves.

3. Settings Walkthrough

Everything below lives on one page: **Settings** (in the Admin section of the sidebar). Each area is its own collapsible section - click a heading to open it. Sections 3.2 (DPD), 3.3 (Email) and 3.6 (My Email) are expanded on below in Sections 4-6, since they need real accounts/credentials set up first.

3.1 Printing

Requires the Print Agent running on the printing PC - see Section 7. Two printers can be set independently: the picking note printer and the shipping label printer (label printer also needs its DPI set to match the physical printer - 203 or 300 dpi). Leave either blank to use that PC's Windows default printer. Without the agent running, picking notes fall back to opening the PDF in a browser tab instead.

3.2 DPD

Covered in full in Section 5 - API credentials, sender address, and customs fields for booking real shipments and printing labels.

3.3 Email

Covered in full in Section 6 - the shared SMTP account used for anyone who hasn't set up their own personal one under My Email.

3.4 Despatch & Packing

- Toggle whether the acknowledgement email and the picking note both fire automatically the moment an order is released for despatch (on by default) - turn off either to keep that as a manual step instead.
- **Packing Mode** - Split Packing (spreads a line's quantity across cartons without tracking which serial went where) or Serial Packing (assigns each individual scanned serial/MAC to a specific carton). Pick one before real despatches start - switching mid-flow isn't supported.

3.5 Returns (RMA)

Two numbers that drive the automatic warranty check on the public RMA form: the non-faulty return window in days (default 28) and the faulty return-to-base warranty in days (default 365/1 year - this is BNS's own RTB warranty; Grandstream's separate warranty still needs checking by hand on their portal).

3.6 Users

Add a real named login for everyone who'll use the system, and change (or retire) the seeded admin password from Section 2.3. There's no admin/staff distinction yet - anyone with a login can create others and change any password.

3.7 My Email

Covered in Section 6 - lets each person send acknowledgement/despatch emails from their own mailbox instead of the shared one.

3.8 Customisation

Per-person order status colours for Sales Activity - purely cosmetic, no setup required, safe to leave on the defaults.

4. Shopify Setup

Two-way integration: pulls in the product catalogue, B2B companies and orders; pushes out despatch confirmation, available stock, and weight. Entirely optional - everything else in the system works without it.

4.1 Create a Shopify app and get credentials

- 1 In the Shopify Partner/Dev Dashboard, create an app for the store (or use an existing custom app).
- 2 Under the app's **App settings** → **Credentials**, note the **Client ID** and **Client Secret**.
- 3 Register a redirect URL on the app that exactly matches: `http://<server>:8080/api/shopify/oauth/callback` - this has to be an exact match (including the port), or Shopify will refuse the handshake.
- 4 Grant the app read access to products, and whatever scopes cover B2B Companies and Orders for your store - the sync itself is read-only for products/companies and only pushes fulfilment/stock/weight updates for orders it has already imported.

4.2 Configure and connect

- 1 In docker-compose.yml (or .env, whichever this deployment uses for these three), set SHOPIFY_SHOP_DOMAIN, SHOPIFY_CLIENT_ID and SHOPIFY_CLIENT_SECRET, then restart: docker compose up --build.
- 2 In the app, go to **Admin** → **Shopify Sync** and click **Connect to Shopify** - this completes Shopify's OAuth flow and stores a real Admin API access token. Reconnecting later (e.g. after revoking access) doesn't need a restart, only the three values above do.

Note: The Connect/OAuth flow deliberately runs on the API's own port (8080), not the app's usual 8081 - this keeps the redirect URL unambiguous. Don't try to "fix" this by pointing it at 8081.

4.3 What each sync does

Sync	Direction	What it does
Products	Pull	Matches by SKU. New products land flagged "Needs Review" (tracking type/default bin have no Sh
Companies	Pull	Name and existence only. Credit limits are never pulled from Shopify - set and stay set here.
Orders	Pull	Every order lands On Hold so credit control and a human review always happen first; matched to a
Stock levels	Push	Available quantity pushed out on a periodic timer - one-way, this system stays authoritative.
Weight	Push	Pushed the moment it's edited here. A Shopify-side edit is overwritten on the next push rather than
Despatch confirmation	Push	Marks the Shopify order fulfilled without triggering Shopify's own customer email - this system's own

Any product Shopify sync brings in flagged "Needs Review" shows up in a list on the Shopify Sync page itself - open each and set its tracking type (and default bin, if it has one) to clear it before stock can be received against it properly.

5. DPD Setup

Route: **Settings** → **DPD**. Needed for booking real DPD shipments and printing real shipping labels from an order - without it, despatch still works, just with a placeholder sample label instead (see the checkbox at the end of this section).

5.1 API credentials

- **API key (Client-Id) / API secret** - from **My DPD** → **API Access** in DPD's own portal. This is a separate developer credential, not your normal DPD login.
- **Environment** - Sandbox for testing (no real shipments/charges), Live once ready to book real collections.
- **Preferred service** (optional) - a DPD service code (e.g. 1^06 for Next Day). DPD always looks up genuinely available services per shipment first; this is only used if it happens to be one of the options offered, so it's a preference, not a guarantee.

- **Freight weight cap** (optional) - a heavy order can tip DPD's service lookup into only offering a pallet/freight service even when it's really going out as ordinary parcels. Setting a cap here (start around 19.9kg and adjust) stops Freight being offered at all for the purposes of that lookup - the real declared weight at despatch is never affected.

5.2 Sender / collection address

Used as both the collection address and the exporter details on any customs declaration. Only Address Line 1 (street), Address Line 3 (town) and Country Code are mandatory - the rest (organisation, contact name/phone, address lines 2 and 4, postcode) fill in the rest of the label. There's no sender email field - DPD's own shipment API doesn't accept one.

5.3 Customs declaration

Needed for shipments to a customs country (currently Ireland). **Goods description** (max 45 characters) is mandatory on every such shipment - DPD reject the booking without it, and warn that vague descriptions get parcels delayed or returned, so be specific (their own example: "women's cotton dresses" rather than just "clothing"). Also set **EORI number** (required for Ireland), **VAT number** (optional), and the declared **Currency** - the customs value itself is calculated automatically from the order's goods lines only, excluding VAT and shipping.

A per-order EORI/VAT override exists too, for shipments where the receiving company is the importer of record rather than BNS - set under that Company's own EORI/VAT fields on the Companies page, and it takes priority over the sender-level values here for that company's shipments.

5.4 Extended liability and fallback labels

- **Extended liability cover** - a chargeable DPD insurance option. Check with your DPD account manager whether this is already included in your account terms before turning it on. When enabled, the insured value sent is the order's goods value, capped at DPD's £5,000 maximum.
- **Print a placeholder sample label when no DPD shipment could be booked** - on by default so despatch never blocks while DPD is still being set up. Turn this off once DPD is fully live, so a failed booking fails loudly instead of quietly printing a fake label.

Note: A white-label / no-return-address template is a DPD account-level setting, not something this integration controls - ask your DPD account manager to configure it on their side if needed.

6. Email Setup

Two layers, both under Settings: a shared fallback account, and an optional per-person override. Leave both blank to run with no email actually sending - acknowledgement/despatch emails still compose and display their content honestly marked "not sent", rather than pretending to.

6.1 Settings → Email (shared/fallback)

SMTP host, port (587 is the usual default), username, password, and the From address everything sends as when no one has their own personal setup below. Changing this takes effect on the very next email sent - no restart needed.

6.2 Settings → My Email (per-person, optional)

Each logged-in person can set their own SMTP username/password, From address, and an optional CC address - acknowledgement and despatch confirmation emails they personally send then go out through their own mailbox instead of the shared one. Leave any one field blank and that part falls back to the shared account. Nothing here is required - it's purely so emails can appear to come from the actual person handling the order.

7. Printing (Print Agent) Setup

A browser can't silently pick a printer or skip the print dialog - that's a deliberate browser security restriction, not a limitation of this app. The Print Agent is a small trusted program that runs on the actual printing PC and accepts print jobs from the browser over localhost, bypassing the dialog entirely. It runs as an icon in the Windows system tray - right-click it for **Settings** (set the SumatraPDF location from a window - no file editing) and **Exit**. Full detail lives in print-agent/README.md in the project; summarised here.

7.1 Which way to run it

Option A - the standalone .exe (recommended for the warehouse PC). A single BNSPrintAgent.exe with the tray icon and Settings window built in - nothing to install on the PC that runs it, not even Python. Someone with Python builds it once (print-agent\build_exe.bat, which bundles the icon and produces dist\BNSPrintAgent.exe); after that the .exe is just copied wherever it's needed. **Option B - running the script directly** (python agent.py) - same tray icon and Settings window as long as pystray and Pillow are installed; without them it still runs, just as a plain console window with no tray icon, exactly as the very first version of this agent did.

7.2 Setup steps

- 1 Install Python on the printing PC (Windows 10/11 usually already has it - any recent 3.x from python.org is fine if not), then install this folder's dependencies: pip install -r requirements.txt (pulls in pystray + Pillow for the tray icon, pywin32 for DPD labels, and pyinstaller if building the .exe).

- 2 Install SumatraPDF from sumatrapdfreader.org/download-free-pdf-reader (free, portable build, no admin rights needed). This is what actually talks to the printer silently, using its `-print-to / -silent` command-line flags - regular Windows PDF viewers don't have a reliable no-dialog print option. The agent expects it at the default install path `C:\Program Files\SumatraPDF\SumatraPDF.exe`; if it ends up somewhere else, set the correct path from the tray icon itself: right-click it → **Settings** → **Browse...** to the real SumatraPDF.exe → **Save**. Takes effect on the very next print, no restart needed - it's saved to `%APPDATA%\BNSPrintAgent\config.json` so it only needs setting once per PC. (The old `SUMATRA_PATH` environment variable still works as a fallback, but the Settings window takes priority over it.)
- 3 Install pywin32 if DPD shipping labels will be printed from this PC - already covered by step 1's requirements.txt on Windows, or on its own with `pip install pywin32`. Not needed for picking notes alone. This is what lets the agent hand raw ZPL label data straight to the label printer at its real physical size, rather than being scaled to a page the way a PDF or HTML label would be.
- 4 Run it - either the built .exe (double-click BNSPrintAgent.exe) or `python agent.py`. A printer icon (with a little person behind it) appears in the system tray - click the ^ arrow in the taskbar if it's hidden there, and optionally drag it out so it's always visible.
- 5 In **Settings** → **Printing** (Section 3.1), enter the exact Windows printer name for the picking note printer and/or shipping label printer (Settings → Printers & Scanners on the PC shows the exact name to copy - for a USB Zebra, this is whatever name Windows gave it when the driver was installed). Leave a field blank to use the PC's default printer instead.
- 6 Test: print a picking note from any order, then book a DPD shipment and print its label. Picking notes fall back to a browser PDF tab if the agent isn't running, so that's never a dead end; DPD labels need the agent running, since a browser can't render or print raw ZPL data itself.

To start the agent automatically at Windows login, put a shortcut to BNSPrintAgent.exe (or, running the script directly instead, a shortcut to `agent.py` / a small .bat file running it) into the Startup folder (shell:startup in the Windows Run dialog).

7.3 Print Agent Troubleshooting

- **"SumatraPDF not found"** - open **Settings** from the tray icon and check the path, or use **Browse...** to find it properly.
- **"Raw label printing needs the pywin32 package"** - run `pip install pywin32` on the PC running the agent, then restart it (only relevant running `agent.py` directly - it's bundled into the .exe).
- **No tray icon, agent runs in a console window instead** - `pystray`/`Pillow` aren't installed; run `pip install pystray pillow` and restart `agent.py` (not applicable to the built .exe, which always has them bundled in).
- **Nothing prints, no error** - check the printer name in Settings exactly matches what Windows calls it; case and spacing matter to some print drivers.
- **DPD labels print but at the wrong size, or as garbled text/barcodes** - the printer isn't a ZPL-compatible label printer, or its Windows driver is intercepting and reprocessing the raw data instead of passing it straight through. Check the driver has a "raw"/pass-through mode if offered, and confirm the DPI setting under Settings → DPD matches the printer (203 vs 300 dpi) - a mismatch there can also throw off barcode scaling.

- **Browser can't reach the agent** - it only listens on localhost, so it must run on the same PC as the browser tab doing the printing. On a shared terminal, the agent needs to run on that terminal specifically.

8. Handheld / Android Kiosk (Optional)

The handheld scanner screens (Picking, Goods In, Stock Movement) work in any browser at /handheld with no extra setup - a phone or tablet browser is enough to try it. The Android kiosk app is only needed for a dedicated, locked-down warehouse scanner device (no browser chrome, can't be swiped away to another app).

- Source lives in android-app/ - it's a real Android Studio project, not yet compiled into an installable APK (that step needs internet access to fetch the Android SDK/Gradle, which the build environment this was generated in doesn't have).
- Before building: edit WAREHOUSE_URL in MainActivity.kt to the server's real LAN address, e.g. `http://192.168.1.245:8081/handheld`.
- Open the android-app folder in Android Studio, let it sync, then Build → Build APK(s) for a quick debug build, or Build → Generate Signed Bundle/APK for a real production install.
- Once a built APK is dropped into android-app/releases/ on the server, it's immediately downloadable from the handheld login page - no container rebuild needed.

9. Bulk Import Files

Two separate, reusable bulk-import tools live under **Settings** - both use the same preview-then-confirm pattern (nothing is written until you explicitly confirm), and both stay available permanently for re-runs, not just a one-off go-live step.

9.1 Bulk Stock Import

Route: Settings → Bulk Stock Import. A one-off replacement of current on-hand stock from an OrderWise export.

File: one spreadsheet (.xlsx/.xls/.csv) with a header row containing these exact columns (case/spacing-insensitive):

Column	Contains
BinNumber	The bin/location code the stock is physically in
VariantCode	Matched against each Product's own SKU - case-insensitive
BatchNo	Batch/carton code for that stock line (optional)
SerialNo	MAC/serial identifier, only for rows where quantity is exactly 1 and it's a genuinely serial-tracked unit (optional)
Qty	Quantity at that bin for that SKU

What it does:

- Matches VariantCode to an existing Product by SKU - a code with no matching product is skipped and listed, never auto-created as a bare product.
- Creates any bin/location codes in the file that don't already exist.
- **Replaces** every stock item currently Available or Quarantined system-wide - despatched history and anything already Allocated to an open order are never touched, whether or not their product appears in the file. This is why it's described as a "go live" seeding operation, not an everyday sync.
- Sets each product's tracking type from the file itself (MAC if any row for that SKU carries an identifier, otherwise NONE) - this overrides whatever tracking type the product currently has.

Caution: Because this replaces on-hand stock wholesale, run it before real orders start flowing through the system, and treat a second run as a full stock recount, not an incremental top-up.

9.2 Company & Contact Import

Route: Settings → Company & Contact Import. Bulk create/update of Companies and their Contacts from two OrderWise exports at once, matched by account code.

File 1 - customer list (one row per company):

Column	Adds/updates
Account number	The match key - becomes the Company's Account Number field
Statement name	Company name
On hold	Credit hold flag - stop processing new orders until cleared
Do not use this account	Do Not Use flag
GAPS	GAPS flag (kept for a report to be built later)
GDMS	GDMS flag (kept for a report to be built later)
Credit limit	Credit limit (leave blank on a row to leave an existing value untouched)
EORI number	Company's own EORI number (used on customs declarations, Section 5.3)
VAT number	Company's own VAT number

File 2 - customer contact details (one row per person, multiple rows per company allowed):

Column	Adds/updates
CustomerCode	The match key - matched against File 1's Account number to link the contact to its company
CustomerContact	Contact's name
ContactEmail	Contact's email
ContactTelephone	Contact's phone
ContactPosition	Contact's job title/position
MainContact	Main-contact flag for that person at that company

ActiveContact	Active flag - blank/unset is treated as active, only an explicit "False" deactivates a contact
---------------	--

Note: The contacts export also has a column literally called **"Main contact"** (with a space) - this is deliberately left unmapped and ignored, separate from **MainContact** (no space) above, because its meaning couldn't be confirmed from sample data. Don't rely on it.

What it does:

- Creates a new Company for any Account number not already in the system, or updates the matching existing one (matched by Account Number, not by name).
- Creates or updates a Contact under the matched company for each contact row - matched to an existing contact at that company by email where possible, to avoid duplicates on a re-run.
- A contact row whose CustomerCode doesn't match any company (in this file or already in the system) is **skipped and listed** in the result, never guessed at or attached to the wrong company.
- "Do not use" and "On hold" companies are imported the same as any other - they are never excluded. Use the On Hold / Do Not Use filters on the Companies page afterwards to find them.

Two fields on each Contact (a Shopify customer ID and an invited-to-Shopify date) exist in the data model already but aren't used by this import or anywhere else yet - they're reserved for a future Shopify B2B "Company Contacts" sync and invitation-email feature, not yet built.

9.3 API keys (not a bulk import, but a related one-off setup step)

Route: Settings → API Access. Generates keys for external integrations - currently the public read-only Stock API (for a customer/partner checking live stock programmatically) and the Support Tickets API (for a call-transcription/CRM import service to create and update tickets). A key is shown once at creation and never again, so copy it immediately; it can be deactivated later but not retrieved. See [docs/BNS_Warehouse_Public_API.pdf](#) and [docs/BNS_Warehouse_Support_Tickets_API.pdf](#) for the endpoints each key authenticates against.

10. Backup & Restore

The database itself is not stored inside the project folder - it lives in a Docker-managed volume (bns_postgres_data) tied to that specific Docker installation, so simply copying the project folder to another machine does **not** bring the data with it (see [docker-compose.yml](#)). Settings → **Backup & Restore** is the built-in way around that - one file with everything needed to stand the system back up elsewhere: the full database (every order, product, stock item, ticket, company, and every Settings field) plus the connection secrets that live only in this server's environment and never touch the database - Postgres credentials, the SMTP account, and the Shopify app's Client ID/Secret (see [.env.example](#)).

10.1 Taking a backup

Settings → Backup & Restore → **Download Full Backup**. Downloads a single .zip (dump.sql + secrets.env + manifest.json) straight to the browser - keep it somewhere safe, since it contains real credentials in plain text.

10.2 Standing up a brand new machine from a backup

Secrets can only be applied to a system at the moment its containers start (Docker reads `.env` only when `docker compose up` runs), so restoring a fresh machine is a script, not an in-app click:

- 1 Copy the project (unzipped release or git clone) onto the new machine, without an existing `.env` in it.
- 2 Copy the downloaded backup `.zip` alongside it.
- 3 Run `./restore.sh <backup-file>.zip` from the project root. It writes `secrets.env` out as this machine's `.env` (backing up any existing one first), brings up just the database, imports `dump.sql` into it, then runs `docker compose up --build` for the full stack.

Note: This is exactly the scenario from Section 2, just with real data and settings already in place - first login is whatever Users existed in the backup (not the seeded `admin/ChangeMe123!` account, unless the backup was itself taken before anyone changed it).

10.3 Restoring into an already-running system

Settings → Backup & Restore → **Restore into THIS system**: choose the backup `.zip`, type `RESTORE` to confirm, and it replaces every product, order, stock item, ticket, company and setting currently in this system's database with whatever is in the backup - useful for cloning production data onto a test box, for example. This cannot be undone. Only the database is restored this way - the backup's secrets are shown (key names only, never displayed values) so you know what's in there, but they're never applied automatically to an already-running container for the same reason as above; copy them into this machine's own `.env` by hand afterwards and run `docker compose up --build` if this system also needs them.

Needs `pg_dump/psql` inside the API's own Docker image to work - already included as of this release's `backend/Dockerfile`; rebuild (`docker compose up --build`) if upgrading from an older release.

11. Before Going Fully Live - Known Gaps

This system has been built and tested as a LAN-only tool. A few things were deliberately left loose specifically because of that, and are worth a dedicated pass together with picking a real host if this ever needs to be reachable outside the local network:

- CSRF protection is currently disabled, and CORS accepts any origin.
- Session cookies aren't marked Secure.
- The public RMA form has no rate limiting.
- There's no role-based permissions yet - every login has equal access to everything, including creating other logins.
- Backups (Section 10) are manual/on-demand - there's no automatic schedule yet, so someone still needs to actually click Download Full Backup on a regular basis and keep the file somewhere safe.

None of this blocks a LAN-only launch tomorrow - it's the punch list for whenever this moves beyond that.